

프론트엔드에서 시작해, 서버 흐름까지 확장해가는 개발자 김응서입니다.

사용자의 행동이 화면, API, 데이터, 알림 흐름까지
끊기지 않도록 설계하고 구현합니다.

Profile.



김응서 Kim Eungseo

Software Engineer

☎ 010-2647-2753

✉ ksy_624@naver.com

Education / Experience

2018.03 ~ 2025.08

부산대학교 · IT응용공학과 졸업

2024.07 ~ 2024.08

(주)에버스톤 · IT 스타트업 현장실습

2025.07 ~ 2026. 06

삼성청년SW·AI아카데미(SSAFY) · 14기 Java 전공반 수료

Awards

2024. 10. 08

부산대학교 LINC 3.0 캡스톤디자인 경진대회 동상

2025. 09. 19

2025 신한은행 해커톤 with SSAFY 우수상

2026. 04. 03

삼성청년SW·AI아카데미 2학기 특화 프로젝트 우수상

Certifications

2022. 09. 02

정보처리기사 · 한국산업인력공단

2024. 11. 24

TOEIC (930) · ETS

2025. 09. 19

SQL 개발자 (SQLD) · 한국데이터산업진흥원

2026. 03. 18

TOEIC SPEAKING (IH) · ETS

Skills.

01 Backend

Java와 Spring 기반으로 REST API, 인증, 배치 작업, 알림 흐름을 구현했습니다.

	Java	★★★★★	알고리즘 풀이와 Spring 기반 백엔드 구현에 활용한 주 사용 언어
	Spring Boot	★★★★★	Spring MVC·DI 구조 이해를 바탕으로 한 REST API와 서비스 기능 구현
	Spring Security	★★★★★	JWT 인증/인가 흐름과 Refresh Token 기반 인증 유지 구조 구현
	Spring Batch	★★★★★	정기 알림 작업 분리와 batch insert 적용을 통한 대량 처리 성능 개선
	JPA	★★★★★	도메인 객체 매핑과 서비스 데이터 저장·조회 기능 구현

02 Frontend

JavaScript와 React 기반으로 화면 구현, 상태 관리, 모바일 앱의 인증·API 연동 흐름을 경험했습니다.

	JavaScript	★★★★★	React 기반 개발에 활용하며 클로저·실행 컨텍스트·스코프 등 동작 원리 학습
	React	★★★★★	컴포넌트 기반 화면 구현과 useState·useEffect 기반 상태·흐름 관리 경험
	React Native	★★★★★	React 기반 지식을 활용한 모바일 화면 구현과 Expo 기반 앱 개발 경험
	React Query	★★★★★	서버 요청과 mutation 상태를 분리한 API 연동 흐름 관리 경험
	Zustand	★★★★★	여러 화면에서 공유되는 클라이언트 상태를 단순하게 관리한 경험

03 Database / Cache

관계형 데이터 저장·조회와 짧은 주기의 상태 데이터 관리를 프로젝트에 적용했습니다.

	MySQL	★★★★★	서비스 주요 데이터를 관계형 구조로 저장하고 조건 기반으로 조회한 경험
	PostgreSQL	★★★★★	프로젝트 환경에서 MySQL 외 관계형 데이터베이스를 활용한 경험
	Redis	★★★★★	TTL 기반 상태 데이터와 토큰 관리, 중복 처리 방지에 활용한 경험

04 Infra / Deployment

컨테이너 기반 실행 환경과 배포 서버, 정적·영상 파일 저장 및 접근 경로를 구성했습니다.

	Docker	★★★★★	환경 차이 문제를 줄이기 위한 Dockerfile·docker-compose 기반 실행 환경 구성
	Nginx	★★★★★	리버스 프록시 역할 이해를 바탕으로 한 백엔드 서버 요청 전달 구조 구성
	AWS EC2	★★★★★	백엔드 서버 배포와 실행을 위한 인스턴스 환경 및 포트 설정 경험
	AWS S3	★★★★★	정적 파일과 영상 파일 저장소 구성, CloudFront CDN 캐싱을 통한 리소스 전달 흐름 연동
	Jenkins	★★★★★	빌드 및 배포 자동화를 위한 CI/CD 파이프라인 구성, GitLab 연동 기반 자동 배포 흐름 구축

05 Collaboration / Automation

Git 기반 협업, 이슈·일정 관리, 문서화와 팀 편의성을 위한 자동화 흐름을 경험했습니다.

	Git	★★★★★	커밋 컨벤션과 브랜치 전략을 바탕으로 한 GitHub·GitLab 기반 협업 경험
	Jira	★★★★★	이슈 단위 작업 관리와 GitLab 연동을 통한 프로젝트 일정 추적 경험
	Notion	★★★★★	회의록, 아이디어, 기능 명세, 자료 공유를 위한 팀 문서화 경험
	Figma	★★★★★	화면 흐름과 UI 구조 확인을 통한 디자인·프론트엔드 협업 경험
	n8n	★★★★★	MR 등록과 리뷰 상황을 메신저로 전달하는 협업 알림 자동화 구성 경험

☆☆☆☆☆ 기초 개념 이해 ★☆☆☆☆ 간단한 기능 구현 가능 ★★★☆☆ 주요 기능 구현 가능 ★★★★★ 문제 해결·개선까지 경험 ★★★★★ 실무 수준의 설계·운영

인사이트

최근 30일 사용 순위를 기준으로 구독 상태를 정리했어요.



구독별 사용 현황



최근 사용하지 않은 구독이 2개 있어요

이번 달 구독료

97,697원

최대 42,440원을 아낄 수 있어요

검토가 필요한 결제 내역 3건

내 구독 리스트

+ 추가

편집

	Claude	연간	월 환산 24,917원
	매년 3월 28일 결제		결제만 유지 중이에요
	쿠팡플레이		7,890원
	매월 1일 결제		결제만 유지 중이에요
	더블 광고형 스탠다드		7,000원
	매월 1일 결제		
	ChatGPT		13,000원
	매월 10일 결제		
	넷플릭스		

구독

인사이트

프로모션

MY

<

월 12,800원 절감

디즈니+ 티빙 웨이브 번들

대상 서비스



디즈니플러스, 티빙, wavve

2026.03.29 ~ 2099.12.31

기존 구독료

34,300원

→

변경 후

21,500원

매월 12,800원 절감할 수 있어요.

연간 절감액

153,600원

디즈니플러스, 티빙, wavve 기준으로 1년이면 153,600원까지 절약할 수 있어요.

프로모션 설명

디즈니플러스, 티빙, wavve 조합을 번들로 이용하면 월

혜택 적용하러 가기

애김 결제내역·사용량 데이터 기반 자동 구독 관리 서비스

SSAFY 팀 프로젝트 (우수상) | 2026.02 ~ 2026.03 | BE, INFRA | 5명

Key Features

구독 통합 관리 기능

OTT 서비스·요금제·번들 정보를 조회하고, 사용자 구독 등록·수정·삭제가 가능하도록 구성

사용 이력 기반 프로모션 추천 기능

구독 상태와 체크인 기록을 바탕으로 서비스별 추천 프로모션과 상세 혜택 정보를 제공

사용자 알림 관리 기능

체크인 유도 및 프로모션 안내 알림을 생성하고, 목록 조회·읽음 처리·삭제까지 이어지는 사용자 알림 흐름 구현

Tech Stack

Backend

Java 17, Spring Boot, Spring Security, JPA, Spring Batch

Database

MySQL, Redis

Infra, CI/CD

Docker, EC2, Nginx

Notification

Firebase Cloud Messaging

01 구독 도메인 API 구현

서비스·요금제·번들 조회부터 구독 등록·수정·삭제, 상세 조회, 체크인 이력까지 사용자 구독 관리 흐름을 담당하는 핵심 API 구현

- 구독 목록·상세·등록·수정·삭제 API 구현
- 서비스·요금제·번들 카탈로그 조회 구조 구성
- 결제 내역 기반 가맹점 매핑 조회·확정 API 연동
- 구독 상태 기반 체크인·추천·알림 흐름 연결

02 Spring Batch 기반 알림 파이프라인 자동화

결제일과 사용량을 기준으로 알림 대상자를 선별하고, FCM 발송까지 이어지는 배치 파이프라인 구현

- 결제일 D-3 기준 체크인 대상 선정 Batch 구현
- 추천 프로모션 알림 대상 선정 및 발송 Batch 구현
- 대상 선정과 발송 Job 분리로 재실행 흐름 관리
- Reader·Processor·Writer 구조로 알림 처리 흐름 분리

03 FCM·Redis 기반 알림 상태 관리

푸시 토큰, 알림함, 읽음·삭제 상태와 Redis 기반 중복 방지를 구현해 알림이 사용자 행동으로 이어지도록 관리

- FCM 토큰 등록·갱신 API 및 Firebase Admin 연동
- 알림 목록 조회, 읽음 처리, 삭제 API 구현
- data-only payload와 딥링크 정보 반영
- Redis dedup key 기반 중복 발송 방지

04 Docker/Nginx 기반 배포 환경 구성

팀원 간 실행 환경 차이를 줄이고 백엔드 서버, DB, Redis가 일관된 구조에서 동작하도록 컨테이너 기반 실행 환경 구성

- dev/prod Docker Compose 환경 구성
- Nginx reverse proxy 요청 전달 구조 설정
- Redis, MySQL, 백엔드 환경변수 관리
- 컨테이너 이미지 기반 실행 구조 구성



Troubleshooting

Spring Batch를 도입했지만 기대한 성능이 나타나지 않았던 문제

문제

- 체크인 대상 선정과 알림 발송은 정해진 시간에 반복 실행되는 작업이었기 때문에, 대상 선정과 발송 과정을 분리하고 안정적으로 관리할 수 있는 구조가 필요했음
- 이를 위해 Spring Batch를 도입했고, 배치 구조로 전환하면 처리 성능도 개선될 것으로 기대했음
- 하지만 기존 서비스 로직과 비교한 결과, Spring Batch를 도입한 초기 구현에서는 기대한 수준의 성능 차이가 나타나지 않았음

기존 방식의 한계

- Processor에서 후보마다 체크인 기록과 기존 알림 존재 여부를 DB에서 건별로 조회하고 있었음
- Writer도 chunk 단위로 데이터를 전달받았지만 내부에서는 한 건씩 저장해, 실제 DB 저장은 단건 처리로 수행되고 있었음
- Spring Batch를 적용했더라도 반복적인 DB I/O가 유지되어 배치 처리의 이점을 충분히 활용하지 못했음

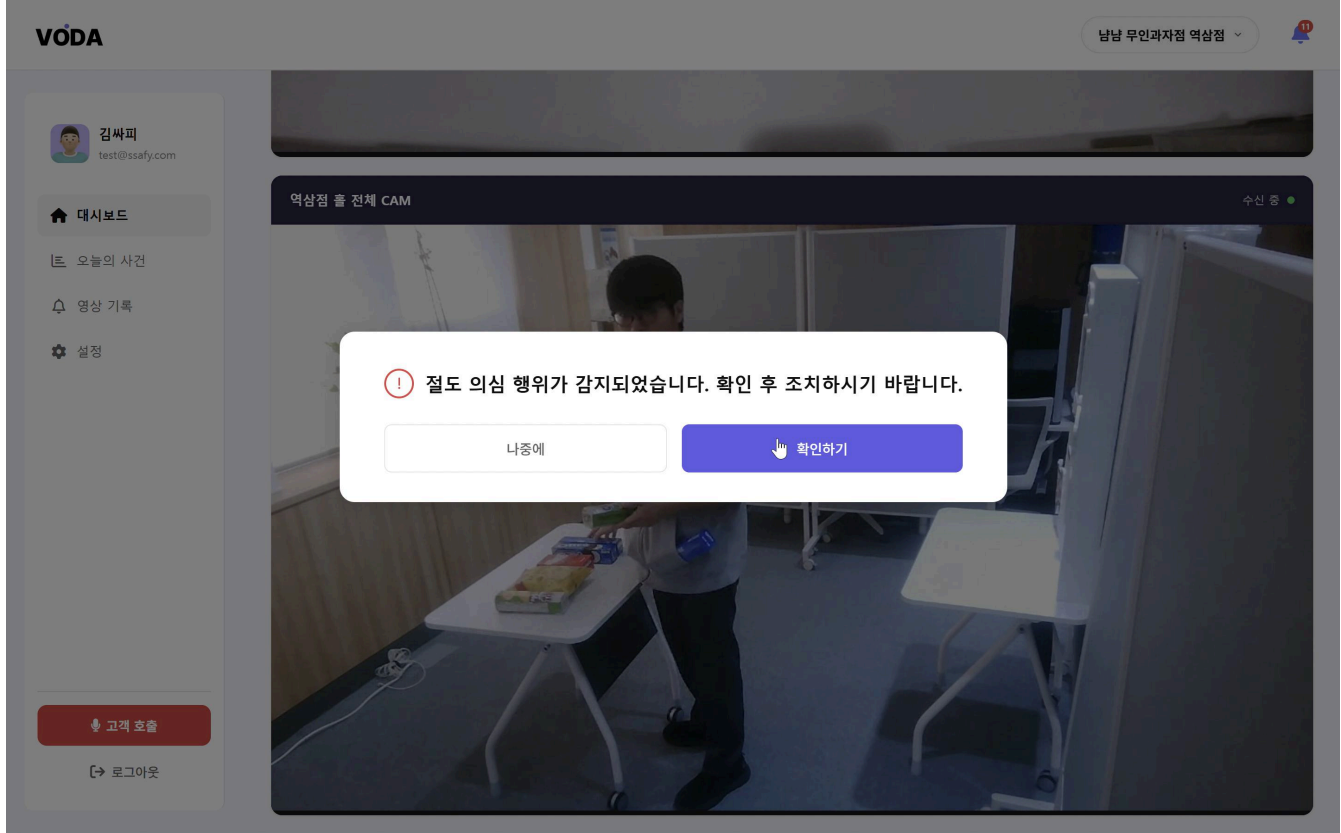
해결

- 체크인 완료 대상과 기존 알림 대상을 한 번에 조회한 뒤, Set으로 구성해 Processor에서 빠르게 비교 하도록 변경
- Writer는 JdbcTemplate.batchUpdate 기반 batch insert로 변경해 chunk 데이터가 실제 DB에서도 일괄 저장되도록 개선
- 체크인 알림 1,500건을 기준으로 개선 후 Batch와 기존 서비스 로직을 각각 30회 측정해 평균 처리 시간을 비교함

결과

- 체크인 알림 1500건 기준, 개선 후 Batch 평균 554ms, 기존 서비스 로직 평균 5514ms로 약 10배 성능 차이를 확인
- Spring Batch 도입 자체보다, 반복 조회와 단건 저장을 줄이는 등 배치에 맞는 내부 구현이 성능에 중요하다는 점을 확인





VODA AI 기반 무인 매장 이상행동 탐지 및 실시간 소통 서비스

SSAFY 팀 프로젝트 | 2026.01 ~ 2026.02 | BE | 6명

Key Features

AI 이상행동 감지 및 실시간 알림

CCTV/키오스크 영상에서 절도 의심 행동을 감지하고 점주 앱에 실시간 알림과 사건 정보를 제공

상품 인식 기반 결제 검증

AI가 인식한 실제 상품과 사용자가 결제하려는 상품을 비교해 누락·불일치 상황을 탐지

사건 타임라인 및 영상 기록 관리

이상행동·미결제 이벤트를 타임라인으로 조회하고, 관련 클립과 전체 영상을 확인할 수 있도록 구성

점주-고객 실시간 소통

오탐이나 고객 문의 발생 시 WebRTC 기반 통화로 점주가 즉시 상황을 확인하고 대응 가능

Tech Stack

Backend

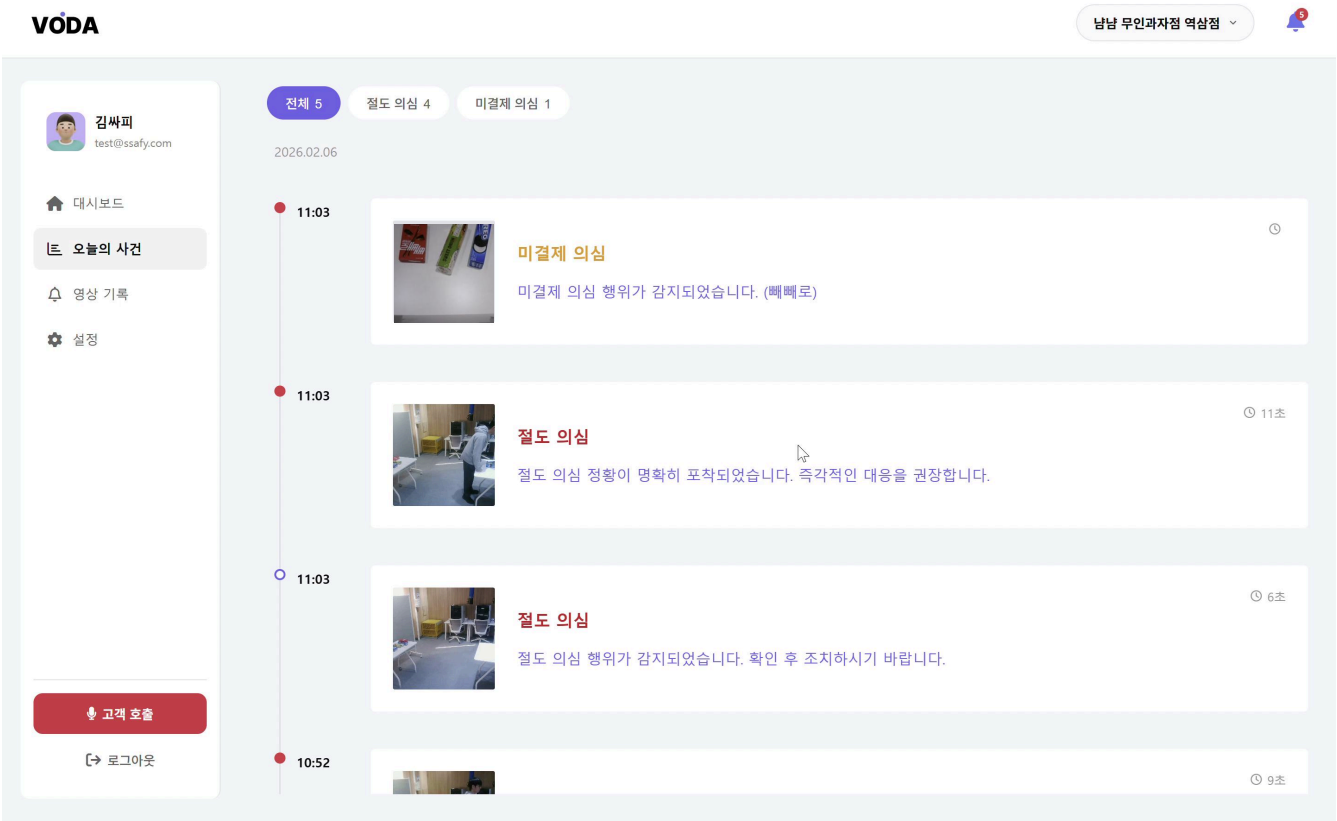
Java 17, Spring Boot, Spring Security, JPA

Database

PostgreSQL

Realtime / Media

WebSocket(STOMP), OpenVidu, AWS S3, CloudFront



My Contribution

AI 감지부터 점주 대응까지 이어지는 백엔드 흐름 구현

01 AI 이벤트 수신 및 실시간 알림 처리

AI 서버에서 발생한 절도 의심 이벤트를 백엔드에서 수신하고, 점주/키오스크 화면에 실시간으로 반영되는 처리 구조 구현

- AI 감지 발생 시점과 영상 저장 완료 시점을 분리해 사건 생성·후속 업데이트 흐름 구성
- AI 신뢰도 점수 기준으로 절도 의심 메시지 강도 분기 처리
- 사건 생성과 동시에 알림을 저장하고, 점주 화면에 실시간 WebSocket 알림 전송
- Spring WebSocket(STOMP) 기반으로 점주/키오스크 알림 채널을 분리해 대상별 메시지 전달

02 결제 분석 세션 기반 상품 검증 흐름 구현

AI 상품 인식 결과와 고객 결제 목록을 비교해 미결제·오인식 상황을 판별하는 결제 검증 흐름 구현

- 결제 분석 세션을 통해 AI 결과와 고객 결제 데이터를 하나의 흐름으로 관리
- 분석 대기 → 결제 검증 완료 → 처리 완료/만료 상태로 상품 분석 진행 상태 관리
- 상품명을 key로 묶어 AI 인식 수량과 고객 입력 수량을 비교하고 부족/초과 상품 판별
- 불일치 결과를 고객 안내 메시지로 변환하고, 키오스크 WebSocket 알림으로 전달

03 영상 기록 및 사건 리플레이 연동

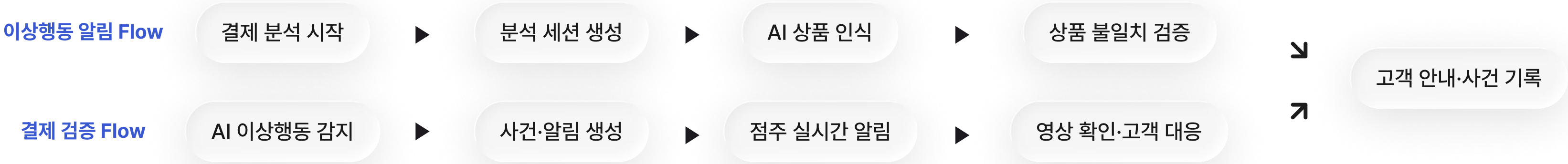
감지된 사건이 영상 기록과 함께 남도록 AI 녹화 결과, S3 저장 경로, 점주용 영상 조회 흐름을 연결

- 영상 녹화 시작/종료 콜백을 통해 영상 기록 생성 및 저장 정보 업데이트
- 결제 검증/이상행동 상황에서 관련 클립 URL을 사건 데이터와 연결
- S3에 저장된 파일 경로(object key)를 CloudFront URL로 변환해 재생 가능한 영상 경로 제공
- 무한 스크롤을 위한 Slice 기반 커서 페이지징으로 사건 타임라인과 영상 기록 조회 구현

04 점주-고객 실시간 통화 및 음성 안내 흐름 구현

AI 오탐이나 고객 문의 상황에서 점주가 즉시 대응할 수 있도록 통화 요청, 상태 관리, 음성 안내 흐름 구현

- 점주-고객 통화 요청을 대기 → 통화중 → 거절/시간초과/종료 상태로 관리
- 진행 중인 통화가 있으면 신규 요청을 제한해 중복 통화 요청 방지
- 통화 수락/거절/종료/부재중 상태를 점주와 키오스크 양쪽 WebSocket 채널에 전송
- TTS로 생성한 안내 음성을 S3에 업로드하고, 점포별 음성 URL로 관리



Troubleshooting

AI 결과와 결제 요청의 도착 순서 차이로 검증 흐름이 끊기는 문제

문제

- AI 결과와 결제 요청이 비동기로 도착하면서, 결제 상품과 비교할 AI 결과·세션 데이터가 준비되지 않아 검증 실패 또는 timeout 발생
- 특히 결제 요청이 AI 결과보다 먼저 도착하면 조회할 분석 세션이 없어 이후 검증 흐름을 이어가기 어려웠음

원인

- 세션 정보가 AI 결과 수신 이후에 생성되어, 결제 요청이 먼저 도착하면 조회할 세션이 없었음
- AI 결과가 항상 먼저 도착한다는 순서를 전제로 설계되어 있었음
- AI 결과와 결제 요청이 동일한 세션 전체를 저장해 서로의 값을 덮어쓸 가능성이 있었음

해결

- 상품 분석을 시작할 때 빈 분석 세션을 먼저 생성하고, AI 서버와 결제 요청에 동일한 세션 ID를 전달
- AI 결과와 고객 결제 정보가 도착하는 순서와 관계없이 같은 세션에 각각 저장되도록 변경
- 두 데이터가 모두 준비되면 상품 수량을 비교해 결제 검증을 수행
- 각 요청이 필요한 값만 갱신하도록 변경해 불필요한 데이터 덮어쓰기를 줄임

결과

- 요청 도착 순서와 관계없이 세션 ID 기준으로 결제 검증 흐름 유지
- 요청 순서 역전 시나리오 100회 재현 결과, 기존 구조에서 발생한 세션 조회 실패 66건이 개선 후 0건으로 감소

점주가 알림 직후 확인하면 영상이 없어 빈 화면만 보이던 문제

문제

- 절도 및 미결제 의심 상황은 점주가 즉시 인지해야 했지만, 실제 영상 클립 URL은 AI 서버의 후처리와 업로드가 끝난 뒤에야 확보 가능했음
- 따라서 점주가 알림을 받고 바로 확인하더라도 영상이 아직 준비되지 않아 빈 화면이 먼저 노출되는 문제가 있었음

원인

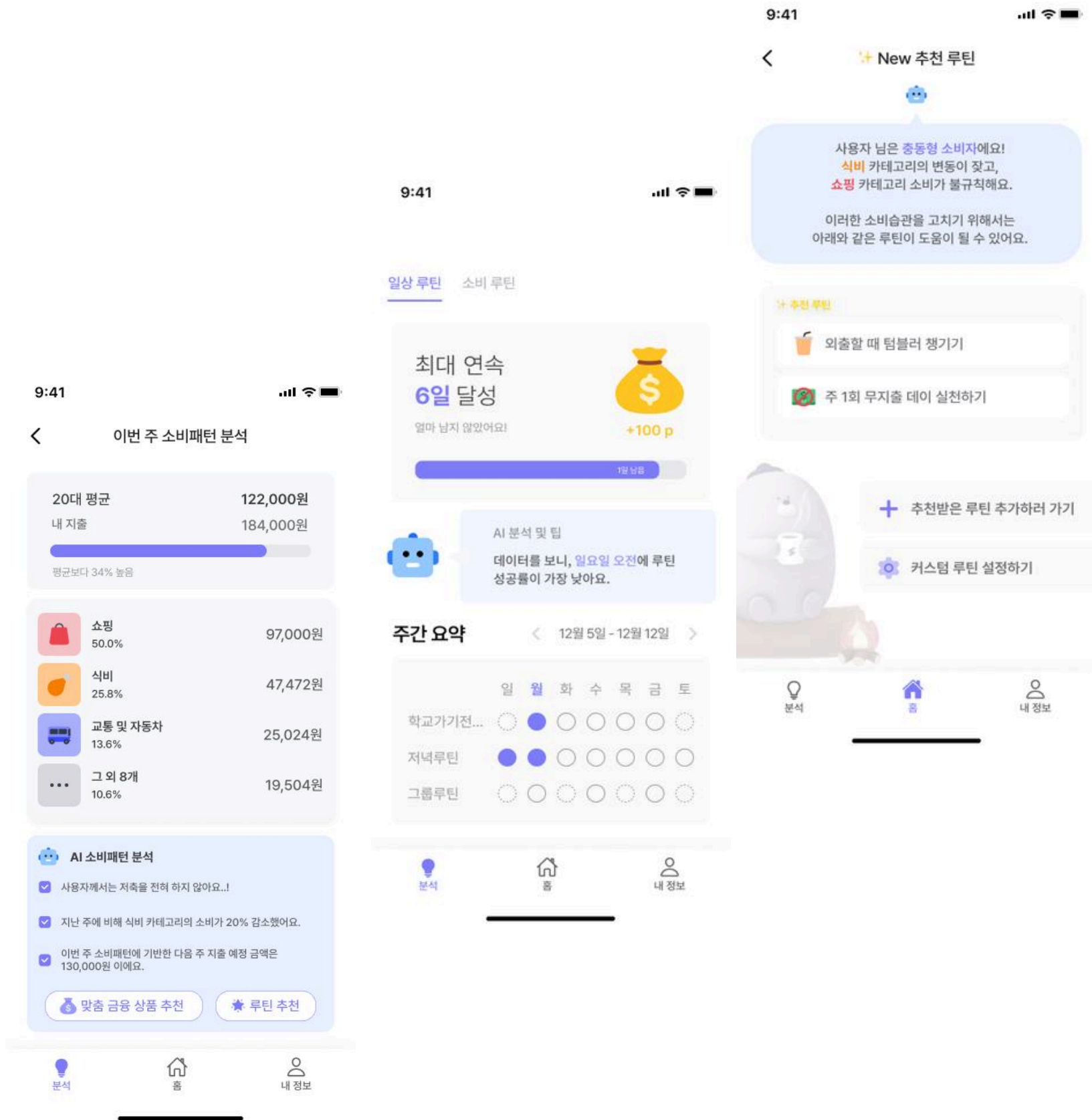
- 이벤트 감지 시점과 영상 클립 생성 완료 시점이 분리되어 있었음
- 영상 URL은 AI 후처리와 S3 업로드가 끝난 이후에만 생성 가능했음

해결

- 이벤트 발생 직후에는 영상 URL 없이 사건 정보만 포함한 1차 알림을 즉시 전송
- 이후 AI 서버가 영상 후처리와 S3 업로드를 완료하면, 백엔드의 후속 결과 수신 API에서 클립 URL을 사건 데이터에 반영
- 이벤트 감지 → 1차 실시간 알림 → S3 업로드 완료 → 영상 링크 반영 → 2차 알림의 단계형 구조로 재설계

결과

- 점주가 이상 상황을 먼저 인지하고, 영상이 준비된 뒤 실제 확인까지 이어질 수 있도록 개선
- 클립 URL이 포함된 2차 알림을 통해 실제 영상까지 확인하는 시간을 이전 대비 5초 이상 단축



HeyRoutine 소비 분석 기반 AI 맞춤 루틴 추천 서비스

신한 해커톤 with SSAFY (우수상) | 2025.08 | FE | 5명

Key Features

신한은행 금융망 API 기반 맞춤 추천

신한은행 금융망 API를 활용한 사용자 소비 데이터 분석과 개인별 루틴 및 금융 상품 추천 기능

개인·단체 루틴 관리

개인 루틴 생성, 수정, 수행 기록과 단체 루틴 참여 및 관리 기능 제공

그룹 루틴 커뮤니티

같은 목표를 가진 사용자들이 단체 루틴에 참여하고, 방명록을 통해 소통할 수 있는 커뮤니티 기능

포인트 리워드 시스템

루틴 수행 결과에 따른 포인트 적립과 포인트 현금화, 기프티콘 교환을 통한 리워드 기능

Tech Stack

Frontend

React Native, Expo, TypeScript, Styled-components, React Query, Zustand

API / Notification

Axios, Firebase Cloud Messaging

My Contribution

인증, 루틴, 금융 기능을 연결한 모바일 프론트엔드 구현

01 금융 인증·계좌 연동 플로우 구현

회원가입, 로그인, 마이페이지, 계좌 등록과 1원 송금 인증까지
사용자 진입과 금융 연동에 필요한 인증 흐름 구현

- 로그인·회원가입·마이페이지 관련 API 연동
- 계좌 등록 및 1원 송금 인증 UI/API 연동
- 토큰 재발급 인터셉터와 인증 예외 처리 적용

02 개인·단체 루틴 기능 구현

개인 루틴과 단체 루틴의 생성, 수행, 기록 확인까지
루틴 실천 흐름을 모바일 화면으로 구현

- 루틴 생성·수정·상세·수행 플로우 구현
- 템플릿, 이모지 선택, 요일·시간 선택 모달 구현
- 개인·단체 루틴 리스트·상세·기록 API 연동

03 분석·포인트 기능 및 공통 구조 정리

소비 분석, AI 추천, 포인트샵 기능을 화면에 연결하고
React Query·Zustand 기반 앱 상태 관리 구조를 정리

- 소비패턴 분석, 금융상품 추천, AI 루틴 추천 화면/API 연동
- 포인트 현금화, 포인트샵 상품 목록·상세·결제 흐름 구현
- React Query·Zustand 기반 요청/클라이언트 상태 분리

Troubleshooting

계좌 인증 재전송 상태가 이전 인증 흐름과 충돌하던 문제

문제

- 인증번호를 재전송해도 이전 입력값·남은 시간·오류 메시지가 유지돼 새로운 인증 시도인지 구분하기 어려웠음
- 계좌 입력·인증번호 요청·검증 상태가 한 화면에서 함께 관리되어 상태 전환 기준이 불명확했음

원인

- 요청·검증·재전송 상태가 한 화면에서 함께 관리되어 재전송을 독립적인 인증 시도로 초기화하지 못함

해결

- 인증번호 재전송 성공 시 입력값·타이머·오류 상태를 함께 초기화하여 새로운 인증 시도 상태로 전환
- 계좌번호 형식을 먼저 검증한 뒤 인증 요청을 보내도록 순서 정리
- React Query의 mutation으로 인증번호 요청과 검증 로직을 분리해 각 요청의 성공·실패 상태를 독립적으로 관리

결과

- 재전송 이후 화면이 새로운 인증 시도 상태로 전환되도록 개선
- 요청 상태와 화면 상태의 기준을 분리해 인증 흐름을 명확하게 관리